Studio Nx

Backend(FastAPI/Python) 68 — get_dashboard()

2026-10-12

Backend(FastAPI/Python) 68 — `get_dashboard()` (대시보드 응답 조립)

항목	내용
파일	`studionx/backend/app/services/dashboard_service.py`
작업	P1-16 대시보드 API: 마이페이지 전체 응답 만들기 (PRD 01 FR-121, 128, 130, 131, 136)
작성	Claude (AI), 2026-10-12, 사용자 검토
이전 · 다음	67 `get_ready_projects()` · 69 `GET /me/dashboard` 라우터 (다음)

1. 하는 일

63~67번에서 만든 조각들을 **한 번에 묶어서** `DashboardResponse` 하나로 돌려줍니다. 새로 계산하는 값은 없고, 프로필 칸만 이 함수가 직접 채웁니다.

응답 칸	만드는 곳
`user`	이 함수 (닉네임, 이메일, 가입일, 사진 주소)
`storage`	64 `get_storage`
`ai`	65 `get_ai_usage`
`ready_projects`	67 `get_ready_projects`
`recent_projects`	66 `get_recent_projects`

비밀번호 해시나 `role`, `status` 같은 나머지 `User` 칸은 응답에 넣지 않습니다. 칸을 하나씩 골라 넣기 때문에, 나중에 `User`에 칸이 늘어나도 응답에 저절로 섞여 나가지 않습니다.

2. 언제 쓰나

호출하는 곳	화면
69 `GET /me/dashboard` 라우터(예정)	myPage.html 전체 (P1-17)

사진 주소는 P1-08에서 만든 `user_service.profile_image_url`을 그대로 씁니다. 그래서 회원 정보 화면과 마이페이지의 사진 주소가 항상 같은 규칙으로 만들어집니다. 사진이 없으면 `None`이고, 화면은 닉네임 첫 글자로 동그라미를 그립니다.

3. 코드

import (파일 맨 위, 이번에 추가한 것만)

```
from app.schemas.dashboard import (
    ...
    DashboardUser,      # 추가
    ...
```

```
)
from app.services.user_service import profile_image_url   # 추가
```

함수

```
def get_dashboard(db: Session, user: User) -> DashboardResponse:
    return DashboardResponse(
        user=DashboardUser(
            nickname=user.nickname,
            email=user.email,
            created_at=user.created_at,
            profile_image_url=profile_image_url(user),
        ),
        storage=get_storage(user),
        ai=get_ai_usage(db, user),
        ready_projects=get_ready_projects(db, user),
        recent_projects=get_recent_projects(db, user),
    )
```

문법 풀이

식	뜻
`DashboardResponse(user=…, storage=…, …)`	Pydantic 모델을 칸 이름으로 만듭니다. 칸이 빠지거나 타입이 틀리면 이 자리에서 오류가 납니다
`DashboardUser(…)`	`user` 칸의 하위 모양입니다. `User` 객체를 통째로 넘기지 않고 네 칸만 골라 넣습니다
`profile_image_url(user)`	저장 경로 `profiles/3f9c….png`를 화면용 주소 `/api/v1/profile-images/3f9c….png`로 바꿉니다

4. 흐름

1. 프로필 네 칸을 `DashboardUser`로 만듭니다 (사진 주소 변환 포함).
2. `get_storage`, `get_ai_usage`, `get_ready_projects`, `get_recent_projects`를 차례로 부릅니다.
3. 다섯 조각을 `DashboardResponse` 하나로 묶어 돌려줍니다.

5. 왜 이렇게

선택	이유
조립만 하는 함수로 분리	각 조각은 63~67번에서 따로 시험을 마쳤습니다. 이 함수는 "조각을 빠짐없이 올바른 칸에 넣는지"만 확인하면 됩니다
`profile_image_url`을 재사용	사진 주소 규칙(회원 번호를 주소에 넣지 않음)을 한 곳에서만 관리합니다
칸을 하나씩 골라 넣음	해시 등 민감한 값이 응답에 섞일 가능성을 구조적으로 없앱니다 (`to_me_response`와 같은 원칙)
DB 조회를 이 함수에서 직접 하지 않음	조회는 각 `get_*`가 맡고, 쿼리 수도 그쪽에서 이미 줄여 두었습니다

6. 시험 (`tests/test_dashboard_service.py`에 5개 추가, 모두 21개)

실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다.

시험	확인한 것
프로필 블록	닉네임, 이메일, 가입일이 맞고, 사진 주소가 `/api/v1/profile-images/<파일명>`입니다
사진이 없음	`profile_image_url`이 `None`입니다
최상위 칸과 비밀 값	칸이 `user`, `storage`, `ai`, `ready_projects`, `recent_projects` 다섯 개뿐이고, `user` 안에는 네 칸뿐이며, `password_hash`가 응답 어디에도 없습니다
조각 합치기	저장 사용량 42, AI 한도 5/30, 최근 프로젝트 순서(New → Old), 이어 하기 2개가 모두 맞습니다
빈 계정	프로젝트가 없으면 두 목록이 모두 빈 목록입니다

전체 시험 517개가 통과했고, `ruff` 검사도 통과했습니다.